Software Engineering Project May-2026

Milestone 3 Deliverables for ZenStudy

Submitted By

Team Name: DeerInTheCode

Team Code : Team-051



IITM Online BS Degree Program,
Indian Institute of Technology, Madras,
Chennai Tamil Nadu, India, 600036

Community Services Platform for Library Management

Project Details

Software Name: ZenStudy

Department Context: Library Management System

System Type: Small Business Logistics & Community Services Platform

Date of Submission: 02-08-2026

Team Details

Team Name: DeerInTheCode
Team Code: Team-051

Name	Roll Number	Role
DIKSHA SHARMA	23f2004899	Product Manager
KARTHIK V N	23f1000033	Frontend Developer
NEHA CHACKO	24f1002416	Scrum Master
GAURAV TOMAR	23f1000805	Backend Developer
AKSHATHA A	23f1001878	Code Reviewer/Tester

API TEST CASES

Test Case 1

- **API being tested**: POST /api/auth/login
- **Inputs:** {"username": "admin", "password": "admin123"}
- **Expected Output:** Status: 200, JSON: {'access_token': '...'}
- **Actual Output:** Status: 200, JSON: {'access_token': '...'}
- **Result**: Success

Snapshot of pytest function

```
def test_login_success(client):
    """Test login with valid credentials"""
    response = client.post("/api/auth/login", json={"username": "admin", "password": "admin123"})
    assert response.status_code == 200
    assert "access_token" in response.get_json()
```

Test Case 2

- **API being tested**: POST /api/auth/login (Invalid)
- **Inputs:** {"username": "admin", "password": "wrongpassword"}
- **Expected Output:** Status: 401, JSON: {'message': 'Invalid username or password'}
- **Actual Output**: Status: 401, JSON: {'message': 'Invalid username or password'}
- **Result:** Success

Snapshot of pytest function:

```
def test_login_failure(client):
    """Test login with invalid credentials"""
    response = client.post("/api/auth/login", json={"username": "admin", "password": "wrongpassword"})
    assert response.status_code == 401
    assert response.get_json()["message"] == "Invalid username or password"
```

Test Case 3

- **API being tested**: GET /api/admin/staff (Unauthorized)
- **Inputs**: No Headers
- **Expected Output**: Status: 401
- **Actual Output**: Status: 401
- **Result**: Success

Snapshot of pytest function:

```
def test_get_staff_list_unauthorized(client):
    """Test getting staff list without token"""
    response = client.get("/api/admin/staff")
    assert response.status_code == 401
```

Test Case 4

- **API being tested**: GET /api/admin/staff (Authorized)
- **Inputs:** Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 200, JSON list of staff
- **Actual Output**: Status: 200, JSON list of staff
- **Result**: Success

Snapshot of pytest function:

```
def test_get_staff_list_authorized(client, admin_token):
    """Test getting staff list with admin token"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/staff", headers=headers)
    assert response.status_code == 200
    assert isinstance(response.get_json(), list)
```

Test Case 5

- **API being tested**: POST /api/admin/staff (Intentionally Failing Test)
- **Inputs**: Headers: {"Authorization": "Bearer <token>"}, JSON: {"username": "newstaff", "password": "password123"}
- **Expected Output**: Status: 201 Created
- **Actual Output:** Status: 400 Bad Request (Missing email)
- **Result**: Fail (Expected 201 but got 400, test catches this error)

Snapshot of pytest function:

```
def test_create_staff_intentionally_failing(client, admin_token):
    """Test creating a staff member - intentionally failing test"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.post("/api/admin/staff", json={"username": "newstaff", "password": "password123"},
                           headers=headers)
    # Intentionally asserting a wrong status code for demonstration
    assert response.status_code == 201
```

Test Case 6

- **API being tested**: GET /api/admin/membership-plans
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 200, JSON list of plans
- **Actual Output**: Status: 200, JSON list of plans
- **Result**: Success

Snapshot of pytest function:

```
def test_get_membership_plans(client, admin_token):
    """Test getting membership plans"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/membership-plans", headers=headers)
    assert response.status_code == 200
    assert isinstance(response.get_json(), list)
```

Test Case 7:

- **API being tested**: GET /api/admin/settings
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 200, JSON settings object
- **Actual Output**: Status: 200, JSON settings object
- **Result**: Success

Snapshot of pytest function:

```
def test_get_library_settings(client, admin_token):
    """Test getting library settings"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/settings", headers=headers)
    assert response.status_code == 200
```

Test Case 8

- **API being tested**: GET /api/admin/seats
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 200, JSON list of seats
- **Actual Output**: Status: 200, JSON list of seats
- **Result**: Success

Snapshot of pytest function:

```python
def test_get_seats(client, admin_token):
    """Test getting seats list"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/admin/seats", headers=headers)
    assert response.status_code == 200
```

Test Case 9

- **API being tested**: GET /api/profile
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 200, JSON profile object
- **Actual Output**: Status: 200, JSON profile object
- **Result**: Success

Snapshot of pytest function:

```python
def test_get_profile(client, admin_token):
    """Test getting own profile"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/profile", headers=headers)
    assert response.status_code == 200
    assert response.get_json()["username"] == "admin"
```

Test Case 10

- **API being tested**: POST /api/staff/students/register
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}, JSON: {'username': 'student1', 'password': 'password123', 'email': 'student1@library.com', 'full_name': 'Student One'}
- **Expected Output**: Status: 201 Created
- **Actual Output**: Status: 201 Created
- **Result**: Success

Snapshot of pytest function:

```python
def test_staff_register_student(client, admin_token):
    """Test staff registering a student"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.post("/api/staff/students/register", json={
        "username": "student1",
        "password": "password123",
        "email": "student1@library.com",
        "full_name": "Student One"
    }, headers=headers)
    assert response.status_code == 201
    assert "student" in response.get_json()
```

Test Case 11

- **API being tested:** GET /api/staff/students/999 (Not Found)
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 404
- **Actual Output**: Status: 404
- **Result**: Success

Snapshot of pytest function:

```python
def test_staff_student_detail_not_found(client, admin_token):
    """Test getting a non-existent student"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/staff/students/999", headers=headers)
    assert response.status_code == 404
```

Test Case 12

- **API being tested**: GET /api/staff/students/999/verify (Not Found)
- **Inputs**: Headers: {'Authorization': 'Bearer <token>'}
- **Expected Output**: Status: 404
- **Actual Output**: Status: 404
- **Result**: Success

Snapshot of pytest function

```python
def test_staff_verify_student_not_found(client, admin_token):
    """Test verifying a non-existent student's membership"""
    headers = {"Authorization": f"Bearer {admin_token}"}
    response = client.get("/api/staff/students/999/verify", headers=headers)
    assert response.status_code == 404
```

Consolidated Report

```
PS D:\IITM\se\milestone-3\MAY2026-Team-051> python -m pytest test_app.py -v
============================= test session starts ==============================
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: D:\IITM\se\milestone-3\MAY2026-Team-051
plugins: anyio-4.11.0, langsmith-0.10.10
collected 13 items

test_app.py::test_login_success PASSED                                   [  7%]
test_app.py::test_login_failure PASSED                                   [ 15%]
test_app.py::test_get_staff_list_unauthorized PASSED                     [ 23%]
test_app.py::test_get_staff_list_authorized PASSED                       [ 30%]
test_app.py::test_create_staff_intentionally_failing FAILED              [ 38%]
test_app.py::test_create_staff_success PASSED                            [ 46%]
test_app.py::test_get_membership_plans PASSED                            [ 53%]
test_app.py::test_get_library_settings PASSED                            [ 61%]
test_app.py::test_get_seats PASSED                                       [ 69%]
test_app.py::test_get_profile PASSED                                     [ 76%]
test_app.py::test_staff_register_student PASSED                          [ 84%]
test_app.py::test_staff_student_detail_not_found PASSED                  [ 92%]
test_app.py::test_staff_verify_student_not_found PASSED                  [100%]
```

```
================================== FAILURES ===================================
_____________________ test_create_staff_intentionally_failing _____________________

client = <FlaskClient <Flask 'app'>>
admin_token = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJmcmVzaCI6ZmFsc2UsImlhdCI6MTc4NTY4MjQwNiwianRpIjoiOTg0MmIzNzctZWMyYS00YjU3LWEz...NWIiLCJleHAiOjE3ODU2ODMzMDYsInJvbGUiOiJBZG1pbiIsInVzZXJuYW1lIjoiYWRtaW4ifQ.EROHs_9c37oAzLFu9MOqEavEXopXj-Fie4VT536eYjU'

    def test_create_staff_intentionally_failing(client, admin_token):
        """Test creating a staff member - intentionally failing test"""
        headers = {"Authorization": f"Bearer {admin_token}"}
        response = client.post("/api/admin/staff", json={"username": "newstaff", "password": "password123"},
                               headers=headers)
        # Intentionally asserting a wrong status code for demonstration
>       assert response.status_code == 201
E       assert 400 == 201
E        +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code

test_app.py:66: AssertionError
=========================== short test summary info ===========================
FAILED test_app.py::test_create_staff_intentionally_failing - assert 400 == 201
========================= 1 failed, 12 passed in 4.36s =========================
```

USER FEEDBACK

After demonstrating the implemented features, the following feedback was received:

- The authentication system works smoothly and tokens are correctly validated.
- The admin interface for staff management is clear, but users requested adding more filters to easily search for staff members.
- Setting up membership plans is intuitive, but users would prefer a way to temporarily disable a plan rather than completely deleting it.
- The system handles unauthorized access correctly and gives proper error messages.

PLANNED IMPROVEMENTS

In the upcoming sprint, we plan to focus on the following:

- Addressing user feedback by implementing search/filtering in the admin staff management page.
- Enhancing the membership plan settings to support 'active/inactive' statuses.
- Implementation of other endpoints related to payment, complaints and attendance management.
- Integrating the frontend with the backend APIs.
- Refining the error handling to provide more detailed validation errors.